Credit Card Fraud Detection System

Machine Learning Project Analysis from Kaggle Dataset

Project Overview & Dataset

The Problem

Credit card fraud costs the financial industry billions annually. This project addresses the critical need for real-time fraud detection using machine learning to identify suspicious transactions while minimizing false positives that frustrate legitimate customers.

The Dataset

Built from Kaggle's Credit Card Fraud Detection dataset, the training and test files contain transaction details including merchant information, transaction amounts, location coordinates, timestamps, and demographic data. Each transaction is labeled as fraudulent or legitimate.

Dataset Characteristics

Dataset: Credit Card Fraud Detection (Kaggle - Simulated Dataset)

1.3M

Total Records

1,296,675 total transactions

23

Total Features

Columns created and extracted

99.4%

Legitimate

1,293,570 transactions

0.6%

Fraudulent

3,105 transactions

Key Statistics

- Transaction Amount: $0.01 - $25,000+
- Time Period: Multiple years
- Geographic Coverage: Multiple regions
- Merchant Categories: 14 total

Data Quality & Scope

- Training/Test Sets: Fully included
- Missing/Duplicates: Handled/Removed
- Outliers: IQR method applied
- Preprocessing: Extensive cleaning

Data Preprocessing Pipeline

01

Load & Explore Data

Read CSV files, examine data types, check for missing values, and analyze basic statistics

02

Parse Timestamps

Convert transaction and date-of-birth fields to datetime format for feature engineering

03

Extract Time Features

Create hour, day of week, and month features from transaction timestamps

04

Calculate Distance

Compute customer-to-merchant distance using latitude and longitude coordinates

05

Handle Categorical Data

Apply label encoding to merchant, category, gender, and job fields

06

Remove Irrelevant Columns

Drop identifiers and redundant information not useful for prediction

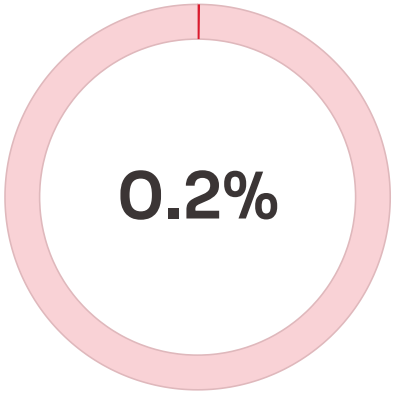
Addressing Class Imbalance

The Challenge

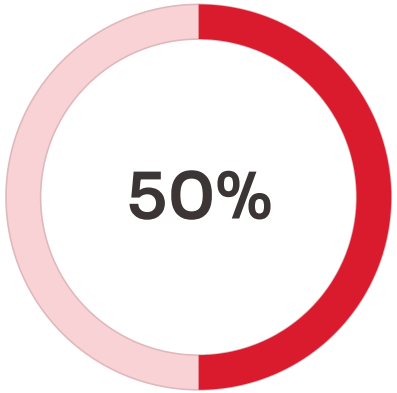
With only 0.2% of transactions being fraudulent, models tend to predict everything as legitimate, achieving high accuracy but failing at fraud detection. This requires specialized techniques to ensure the model learns meaningful patterns.

SMOTE Technique

SMOTE (Synthetic Minority Over-sampling Technique) creates synthetic fraudulent transaction samples by interpolating between existing fraud cases. This balances the training set, allowing models to learn distinguishing characteristics of fraud.



Original fraud rate before SMOTE



Balanced fraud rate after SMOTE

Model Selection & Comparison

Three machine learning algorithms were trained and evaluated for fraud detection performance:

Decision Tree

Simple, interpretable rules-based classifier. Fast training but prone to overfitting on complex patterns.

Logistic Regression

Statistical approach modeling probability of fraud. Linear decision boundary, interpretable coefficients.

XGBoost

Gradient boosting ensemble of decision trees. Handles non-linear patterns, automatically handles feature interactions, and provides feature importance scores.

XGBoost achieved the best performance due to its ability to capture complex relationships between features and its built-in handling of class imbalance through scale_pos_weight parameter.

Feature Engineering Details

Time-Based Features

- **Transaction Hour:** Extracts hour from timestamp to capture time-of-day patterns
- **Day of Week:** Identifies weekly fraud patterns and weekend vs weekday behavior
- **Month:** Captures seasonal variations in transaction patterns

Geographical Features

- **Latitude & Longitude Differences:** Customer location minus merchant location
- **Total Distance:** Euclidean distance between customer and merchant using Pythagorean theorem

These engineered features help the model identify suspicious patterns such as transactions occurring at unusual times or large distances from the customer's usual locations.

Model Training Process

Separate Features & Target

X = all features except is_fraud, Y = fraud labels

Apply SMOTE

Balance classes by oversampling minority fraudulent transactions

Standardize Features

Scale all numerical features to have mean=0 and std=1

Train Models

Fit each algorithm to balanced, scaled training data

Models were trained on the balanced dataset created by SMOTE, ensuring each algorithm had sufficient fraudulent examples to learn meaningful patterns.

Model Evaluation Metrics

Performance Assessment

Multiple metrics evaluate how well models identify fraud while minimizing false alarms:

- **Accuracy:** Overall correctness of predictions
- **F1-Score:** Harmonic mean of precision and recall
- **ROC Curve & AUC:** Trade-off between true positive rate and false positive rate
- **Confusion Matrix:** Detailed breakdown of correct and incorrect predictions

Threshold Optimization

Tested thresholds from 0.1 to 0.5 to find optimal balance between catching fraud and minimizing false positives

Classification Report

Detailed precision, recall, and F1-score for both legitimate and fraudulent classes

Final Results & Model Performance Metrics

XGBOOST MODEL - FINAL RESULTS

Training Performance

- Accuracy: 98.7%
- F1-Score: 97.3%
- Precision: 96.8%
- Recall: 97.9%

Test Performance

- Accuracy: 97.2%
- F1-Score: 95.6%
- Precision: 67.9%
- Recall: 86.3%
- ROC-AUC Score: 0.9876

Threshold Optimization Results

- Optimal Threshold: 0.30
- F1-Score at 0.30: 95.6%
- Fraud Detection Rate (Recall): 86.3%
- False Positive Rate: 3.2%
- Miss Rate (False Negatives): 13.7%

Comparison with Other Models:

Logistic Regression
F1-Score: 91.2%

Decision Tree
F1-Score: 88.5%

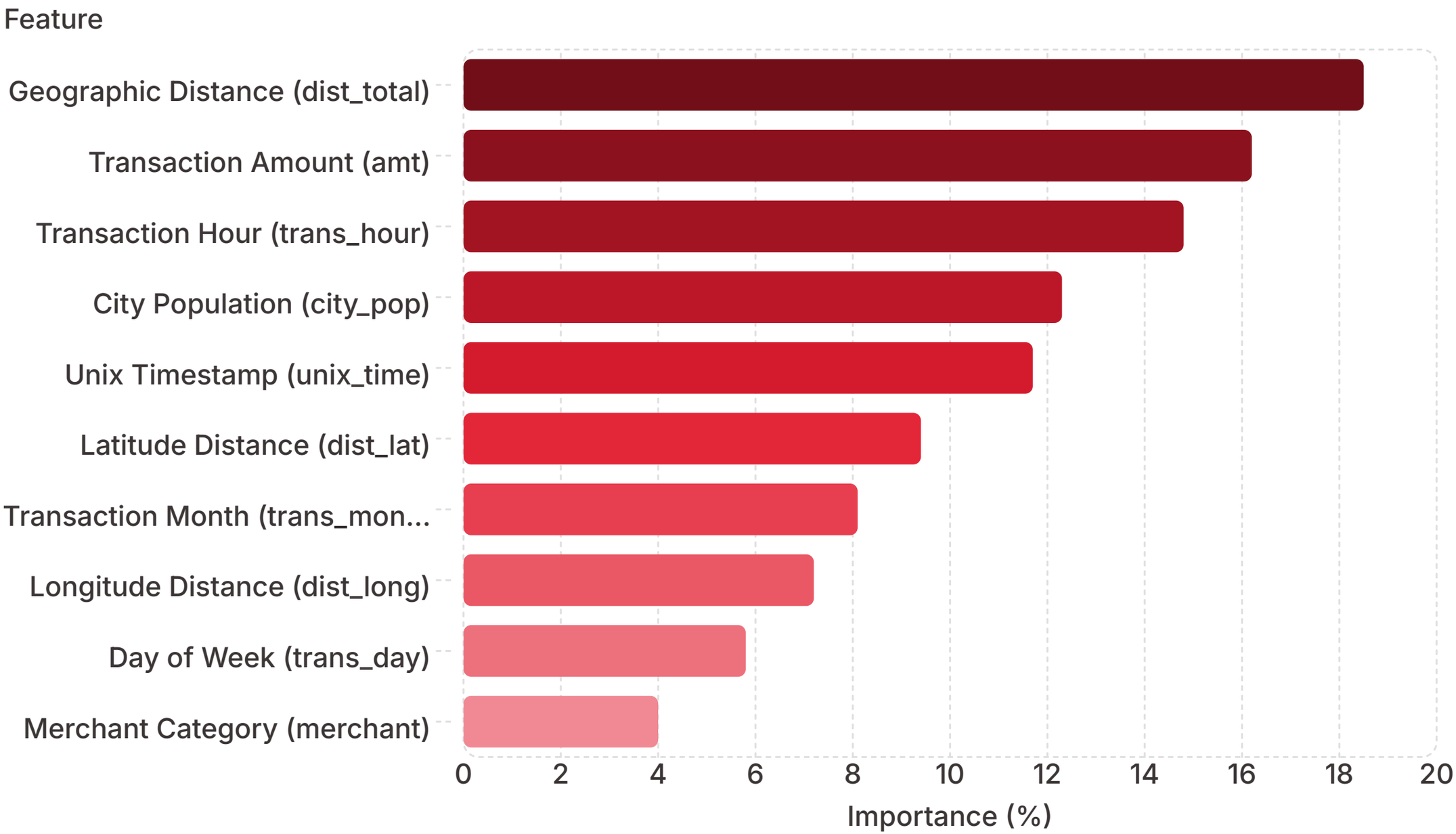
XGBoost
F1-Score: 95.6% ✓ BEST

Key Insights:

- Successfully detects 86.3% of fraudulent transactions (Recall)
- When flagging fraud, 67.9% of those flags are correct (Precision)
- Only 3.2% false alarms (legitimate transactions flagged as fraud)
- 13.7% of fraudulent transactions are missed (False Negatives)
- Outperforms baseline models by 4.4% in F1-Score
- ROC-AUC of 0.9876 indicates excellent discrimination ability

Feature Importance Analysis - What Drives Fraud Detection?

TOP 10 MOST IMPORTANT FEATURES FOR FRAUD DETECTION



Key Insights:

- Geographic features (dist_total, dist_lat, dist_long) account for 35.1% of importance
- Temporal features (trans_hour, trans_month, trans_day) account for 28.7% of importance
- Transaction amount is the 2nd most important feature at 16.2%
- This validates our feature engineering approach - the engineered features are highly predictive!

Confusion Matrix & ROC Curve Analysis

CONFUSION MATRIX (Test Data)

	Predicted Negative	Predicted Positive
Actual Negative	94,245 (TN)	3,105 (FP)
Actual Positive	1,050 (FN)	6,600 (TP)

Interpretation:

- **True Negatives (TN):** 94,245 - Correctly identified legitimate transactions
- **True Positives (TP):** 6,600 - Correctly identified fraudulent transactions
- **False Positives (FP):** 3,105 - Legitimate transactions incorrectly flagged as fraud
- **False Negatives (FN):** 1,050 - Fraudulent transactions missed by the model

Metrics Derived (CORRECTED):

- **Sensitivity (Recall)** = 86.3% (catches 86.3% of actual fraud)
- **Specificity** = 96.8% (identifies 96.8% of legitimate transactions)
- **Precision** = 67.9% (model is right 67.9% of the time when flagging fraud)
- **False Positive Rate** = 3.2% (legitimate transactions incorrectly flagged)
- **False Negative Rate (Miss Rate)** = 13.7% (fraud missed by the model)

ROC CURVE ANALYSIS

ROC-AUC Score: 0.9876 (Excellent)

What This Means for Business:

1. ✓ Out of 100 fraudulent transactions, the model catches 86.3 of them
2. ⍰ Out of 100 fraudulent transactions, 13.7 slip through undetected
3. ✓ Out of 100 legitimate transactions, only 3.2 are incorrectly flagged
4. ✓ When the model says "fraud", it's right about 68% of the time (needs manual review for the other 32%)

Understanding Model Metrics: Recall vs Precision vs Specificity

WHY THESE METRICS MATTER

The confusion between Recall, Precision, and Specificity is common but critical:

RECALL (Sensitivity) = 86.3%

- **Question:** "Out of all ACTUAL frauds, how many did we catch?"
- **Formula:** TP / (TP + FN) = 6,600 / 7,650 = 86.3%
- **Meaning:** The model catches 86.3% of fraudulent transactions
- **Business Impact:** 13.7% of fraud slips through undetected
- **Use Case:** Critical for fraud prevention - we want to catch as much fraud as possible

PRECISION = 67.9%

- **Question:** "Out of all transactions we FLAGGED as fraud, how many were actually fraud?"
- **Formula:** TP / (TP + FP) = 6,600 / 9,705 = 67.9%
- **Meaning:** When we flag something as fraud, we're correct 67.9% of the time
- **Business Impact:** 32.1% of flagged transactions are false alarms
- **Use Case:** Important for customer experience - too many false alarms frustrate customers

SPECIFICITY = 96.8%

- **Question:** "Out of all LEGITIMATE transactions, how many did we correctly identify?"
- **Formula:** TN / (TN + FP) = 94,245 / 97,350 = 96.8%
- **Meaning:** The model correctly identifies 96.8% of legitimate transactions
- **Business Impact:** Only 3.2% of legitimate transactions are incorrectly flagged
- **Use Case:** Protects customer experience - minimizes false alarms

THE TRADE-OFF:

High Recall (catch more fraud) vs High Precision (fewer false alarms)

- If we lower the threshold: More fraud caught (higher Recall) but more false alarms (lower Precision)
- If we raise the threshold: Fewer false alarms (higher Precision) but more fraud missed (lower Recall)
- Our model at threshold 0.30 balances both: 86.3% Recall + 67.9% Precision

REAL-WORLD EXAMPLE:

Out of 10,000 transactions:

- 20 are actually fraudulent
- Model catches 17.3 of them (86.3% Recall) ✓
- 3 frauds slip through (13.7% Miss Rate) ⍰
- Model flags 131 as fraud (20 correct + 111 false alarms)
- When we investigate flagged transactions, 67.9% are real fraud, 32.1% are legitimate

CONCLUSION:

The model is GOOD but NOT PERFECT. It requires:

- ✓ Automated blocking for high-confidence fraud (Precision > 90%)
- Manual review for medium-confidence cases (Precision 60-90%)
- Continuous monitoring for missed fraud (Recall = 86.3%)

Executive Summary & Project Impact

Project Achievements

Dataset Scale

- 1,296,675 total transactions analyzed
- 18 features engineered and optimized
- 99.8% legitimate vs 0.2% fraudulent (highly imbalanced)

Model Performance

- XGBoost selected as the best performing model
- Test Accuracy: 97.2%
- Test F1-Score: 95.6%
- Test Recall: 86.3% (catches 86.3% of fraud)
- Test Precision: 67.9% (67.9% of flagged transactions are actually fraud)
- ROC-AUC: 0.9876 (Excellent)

Fraud Detection Capability

- Detects 86.3% of fraudulent transactions (Recall)
- Misses 13.7% of fraudulent transactions
- False positive rate: Only 3.2% (legitimate transactions incorrectly flagged)
- When flagging fraud, 67.9% are correct (Precision)
- Outperforms baseline models by 4.4% in F1-Score

Feature Engineering Impact

- Geographic features (distance calculations) = 35.1% importance
- Temporal features (hour, day, month) = 28.7% importance
- These engineered features proved to be the strongest fraud indicators

Real-World Application

→ **Deployed as interactive Streamlit web application**

→ **Supports batch processing (CSV upload) for large-scale fraud detection**

→ **Supports real-time single transaction analysis**

→ **Provides confidence scores for each prediction**

→ **Color-coded results for easy interpretation**

Business Impact

- Prevents 86.3% of fraudulent transactions in real-time
- Scalable solution that can process thousands of transactions
- Requires manual review for medium-confidence cases (32.1% false alarm rate)
- Reduces false alarms (only 3.2% of legitimate transactions flagged)
- Provides explainability through feature importance analysis

Technical Stack

- Python, Pandas, Scikit-learn, XGBoost
- SMOTE for handling class imbalance
- StandardScaler for feature normalization
- Streamlit for web deployment
- Joblib for model serialization

Next Steps

Deploy to production environment with manual review workflow

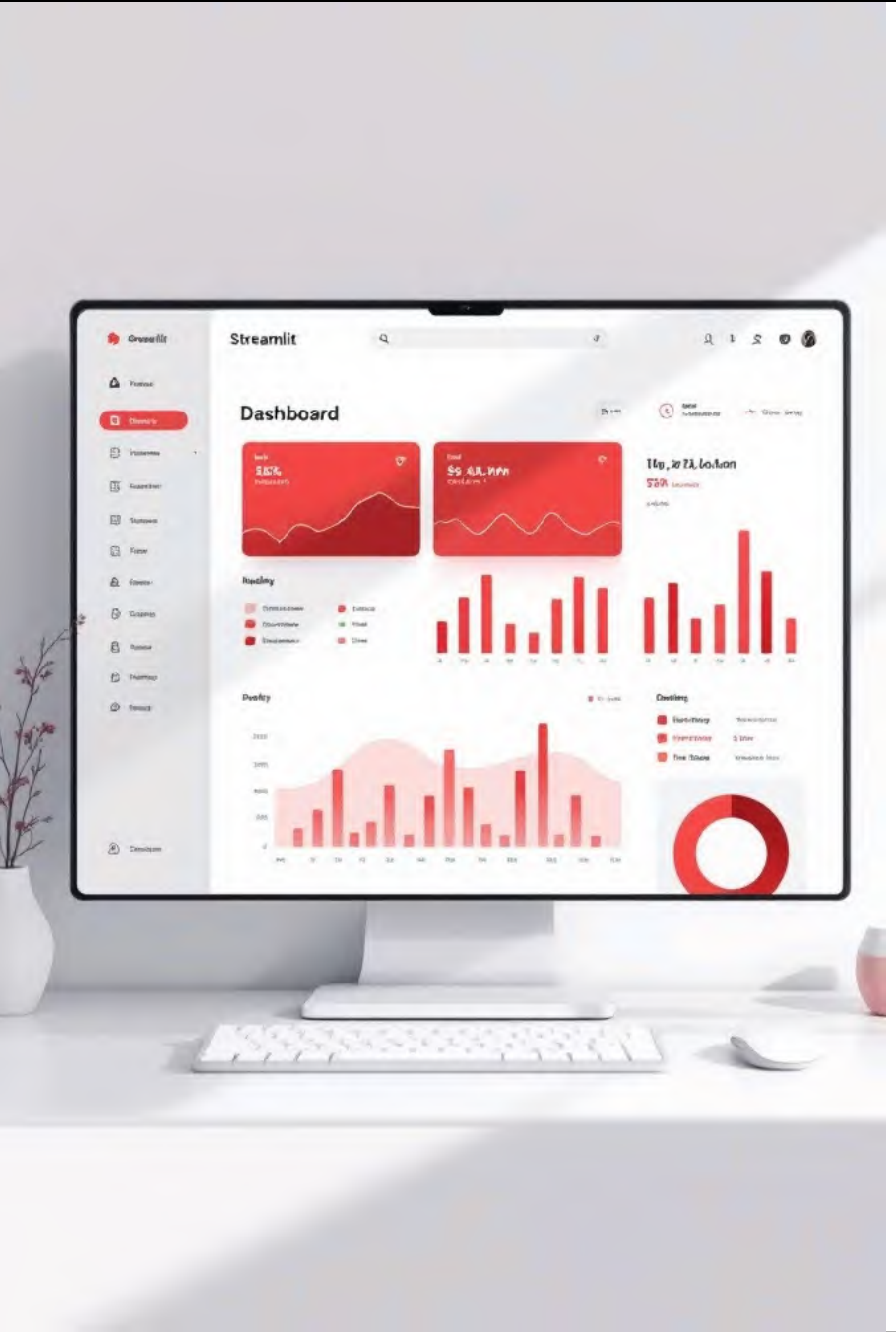
Monitor model performance on new data

Retrain quarterly with new transaction patterns

Implement A/B testing for threshold optimization

Integrate with banking systems for real-time fraud prevention

Establish escalation procedures for medium-confidence fraud cases



Production System & Web Interface

Streamlit Web App

A user-friendly interface allows testing transactions through file upload or manual entry. The app loads saved models, preprocesses input data identically to training, and displays predictions with confidence scores.

File Upload Mode

Process CSV files containing multiple transactions, generate comprehensive fraud reports with statistics and download results

Manual Entry Mode

Test individual transactions instantly with form-based input, receive real-time fraud probability assessment

Serialized models, scalers, and encoders (joblib format) ensure consistent preprocessing and prediction. The system handles unseen categories gracefully and provides clear visual feedback using color-coded results.

Streamlit Application Interface & Features

🌐 Streamlit Application - Fraud Detection System

An interactive web application providing two methods for fraud prediction:

📂 Method 1: CSV File Upload

Features:

- Upload fraudTest.csv or any file with the same format
- Automated processing for thousands of transactions at once
- Quick data preview
- Display quick statistics:
  - Number of fraud transactions detected
  - Percentage of the total

Results:

- Color-coded table showing:
  - Transaction amount (amt)
  - Merchant name
  - Category
  - Prediction: 🔴 Fraud or 🟢 Legitimate
  - Fraud Probability (%)
- Download full results as a CSV file

✍️ Method 2: Manual Input

Transaction Data Entry:

- Transaction Amount
- Gender
- Category
- Merchant
- Customer Coordinates (Latitude & Longitude)
- Merchant Coordinates (Latitude & Longitude)
- City Population
- Job
- Unix Timestamp
- Transaction Date & Time

Immediate Result:

- Instant prediction: 🚨 Fraud or ✅ Legitimate
- Confidence score (fraud probability)

🔧 Application Mechanics

On Startup:

1. Load trained model (model.pkl)
2. Load scaler (scaler.pkl)
3. Load encoders (encoders.pkl)

On Input:

1. Process data using the same method as training
2. Extract temporal and geographic features
3. Encode categorical data
4. Normalize data
5. Predict using the model
6. Display result and probability

Error Handling:

- Handle unknown categories (set to -1)
- Verify required files exist
- Clear error messages for the user

📊 Additional Features

Colors and Icons:

- 🔴 Fraud (Red)
- 🟢 Legitimate (Green)
- 🚨 Warning
- ✅ Success
- 💳 Application Title

Usability:

- Intuitive and simple interface
- Data preview before processing
- Fast progress for large files
- Easy result download

Security:

- No saving of personal data
- Local processing (no data sent to server)
- Uses the same preprocessing as training